How LLMs Fail in Code Generation: A Cross-Benchmark and Unified Fine-Grained Error Taxonomy

Yujun Lu*
School of Software
Nanjing University of Information Science and Technology
Nanjing, Jiangsu, China
YujunLu_Rise@outlook.com

Abstract

Execution-based benchmarks now dominate code-LLM evaluation: EvalPlus densifies HumanEval and MBPP with many more unit tests, while BigCodeBench stresses rich tool use with near-exhaustive coverage, shaping failure profiles beyond mere syntax errors. Prior work often reports Pass@k alone, lacks a unified, cross-benchmark taxonomy, and under-examines how deliberate "thinking" modes affect error composition and failure severity. To address this, we establish a unified, fine-grained error taxonomy that harmonizes labels across different evaluators and analyze the effect of "thinking" mode by macro and micro error ratios. Our work reveals a stable, generalizable profile in which semantic mismatches dominate and A_EQ_NEQ (equality/inequality reasoning) remains the primary bottleneck even with deliberate reasoning. On BCB-hard, DeepSeek-V3.1 (Thinking) shows slightly lower error ratios despite a lower Pass@1, while Qwen-Plus (Thinking) both narrows its Pass@1 gap to a coder-oriented model and achieves smaller error ratios. These findings demonstrate that the proposed error-ratio lens exposes shifts in failure severity that Pass@k alone obscures.

CCS Concepts

• **Software and its engineering** → Software creation and management; Software development techniques; Automatic programming; Software creation and management; Software verification and validation; Software defect analysis; Software creation and management; Software verification and validation; Software defect analysis; Software testing and debugging; • **General and reference** → Cross-computing tools and techniques; Empirical studies.

Keywords

Empirical Study, Code Generation, Large Language Models, Error Classification

ACM Reference Format:
Yujun Lu. 2026. How LLMs Fail in Code Generation: A Cross-Benchmark and Unified Fine-Grained Error Taxonomy. In *The 2nd International Conference on Sustainable Development and Digital Transformation (SDDT 2026), May 29–31, 2026, Xi'an, China.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3838938.3838941

*Corresponding author.

This work is licensed under a Creative Commons Attribution 4.0 International License.
SDDT 2026, Xi'an, China
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2438-1/2026/05
https://doi.org/10.1145/3838938.3838941

1 Introduction

Recent breakthroughs in Large Language Models (LLMs) have dramatically advanced the long-standing goal of automatic code generation from natural language [1-3] https://ar5iv.org/abs/2406.08731-:$\sim$:text$=$Automaticallygeneratingcodefromnatural, Answeringthesequestionswouldhelp. Modern code-focused LLMs can now solve many programming problems and even approach average human performance in coding competitions [4-6] https://arxiv.org/html/2405.04520v1-:$\sim$:text$=$Largelanguagemodels(LLMs)pre, forpracticalproductionandapplication. These advances have driven the adoption of LLM-based coding assistants (e.g., GitHub Copilot) to boost developer productivity [7]. However, current models are still far from infallible – they frequently produce incorrect or buggy code and cannot be fully trusted for all taskshttps://ar5iv.org/abs/2406.08731-:$\sim$:text$=$,identifyopportunitiesformodelimprovement. For example, even the strongest available LLM only achieves about a 53% success rate on a challenging, real-world coding benchmark, underscoring the substantial gap to truly reliable code generation [8]https://arxiv.org/html/2405.04520v1-:$\sim$:text$=$NCB,worldcodingchallenges.

To evaluate LLM code generation, researchers have developed execution-based benchmarks that validate generated code against unit tests, like HumanEval [5], MBPP [9], BigCodeBench [10], LiveCodeBench [11] and DS-1000 [12]. These benchmark suites cover a range of programming tasks but largely focus on relatively simple, well-defined functionshttps://arxiv.org/html/2405.04520v1-:$\sim$:text$=$Notwithstandingtheircommendablereliabilityand, askstoreadandparse. Using such benchmarks, successive LLM iterations (including code-specialized models and general models like GPT-4) have shown steadily improving pass rates on coding taskshttps://ar5iv.org/abs/2406.08731-:$\sim$:text$=$Automaticallygeneratingcodefromnatural, Answeringthesequestionswouldhelp.

While these benchmarks primarily track pass rates, recent studies have begun to delve deeper into the failure modes on a standard Python benchmark and categorized the 12,837 resulting errors into 19 root-cause groups [13]https://openreview.net/forum?id$=$m4mowfglwk-:$\sim$:text$=$thispaper,wefirstconducted,19categoriesoferrorcauses. Another work derived a comprehensive taxonomy of code generation errors via manual analysis of hundreds of faulty solutions, finding

that many mistakes stem from complex logical misunderstandings (e.g., incorrect conditional logic or missing steps) rather than simple syntax bugs [14]https://ar5iv.org/abs/2406.08731-:$\sim$:text$=$CodeGen,syntacticcharacteristicsoftheseerrorshttps://ar5iv.org/abs/2406.08731-:$\sim$:text$=$Ouranalysisshowsthatthese, ofhandlingthediverseset. There have also been efforts to automatically repair erroneous LLM code via static analysis or by using a second LLM as a fixer, but such methods address only a subset of issues since the fundamental causes of many errors remain insufficiently understood [15, 16]https://openreview.net/forum?id$=$m4mowfglwk-:$\sim$:text$=$manyerrors,In.

However, important gaps remain in our understanding of LLM coding failures. Many prior evaluations and error analyses have been limited in scope – typically examining a single model or a single benchmark (often Python-only, such as HumanEval) in isolationhttps://ar5iv.org/abs/2406.08731-:$\sim$:text$=$CodeGen,syntacticcharacteristicsoftheseerrors. This narrow focus therefore obscures whether previously identified error patterns are fundamental to LLM code generation or are merely artifacts of a specific model architecture or benchmark domain. Consequently, the community lacks a reliable compass for directing model improvement efforts. Moreover, most studies evaluate models using straightforward prompting, whereas the effects of advanced inference techniques (e.g., chain-of-thought reasoning) on the types of errors produced have not been thoroughly examined. The benchmarks themselves also have limitations: many widely used test sets consist of relatively basic algorithmic problems, failing to capture the full complexity of real-world software taskshttps://arxiv.org/html/2405.04520v1-:$\sim$:text$=$Notwithstandingtheircommendablereliabilityand, askstoreadandparse. Overall, the literature lacks a broad, cross-model and cross-benchmark perspective on where and why current LLMs fall short in code generation.

To address these gaps, we conduct a large-scale, cross-model and cross-benchmark analysis of LLM code-generation failures on HumanEval+, MBPP+ (EvalPlus), and BigCodeBench (Full/Hard). To enable apples-to-apples comparison, we introduce a unified fine-grained error taxonomy that harmonizes heterogeneous labels across evaluators (e.g., mapping HumanEval+ "WrongAnswer" and BigCodeBench "AssertionError" to consistent semantic classes), and we quantify failure severity with two test-centric aggregates computed strictly from official unit tests: the macro error ratio (unweighted mean of per-task error rates) and the micro error ratio (pooled failed tests divided by pooled total tests). This pairing preserves fidelity to each benchmark's executable tests while providing a common analytical lens over models, datasets, and reasoning modes. (BigCodeBench offers per-task unit tests with high branch coverage; EvalPlus densifies HumanEval/MBPP; macro/micro aggregation follows standard definitions.)

Addressing Gap 1 (scope restricted to single model/benchmark), we deliver a comparative, cross-model/cross-benchmark error anatomy and establish generalizable error patterns that persist across diverse model families and datasets, moving beyond isolated case studies. Addressing Gap 2 (unknown impact of advanced reasoning/CoT on error types), we show that the equality/inequality subclass (A_EQ_NEQ) remains the dominant bottleneck even under deliberate reasoning, and—via our macro/micro error-ratio lens—reveal how reasoning can redistribute failure severity (e.g., lowering the average number of failed unit tests per failed task) without uniformly improving Pass@k. Together, the unified taxonomy and error-ratio analysis constitute a methodological innovation that makes progress measurable at the level where models most often fail while remaining fully reproducible from public evaluators and their official test suites.

The rest of this paper is organized as follows. Section 2 details our methodology, including the benchmarks, models, evaluation pipeline, and the construction of our unified error taxonomy. Section 3 presents the experimental results, analyzing Pass@1 scores, error distributions, and the critical insights revealed by our error-ratio analysis. Finally, Section 4 concludes the paper by summarizing our key findings, discussing their implications, and outlining promising directions for future work.

2 Methodology

We summarize our experimental workflow in Figure 1. The diagram presents the end-to-end pipeline used throughout Section 2: benchmark ingestion; model prompting under non-thinking and thinking modes; official sanitization and unit-test evaluation; failure harvesting; taxonomy-based auto-labeling; and error-ratio computation and reporting. We refer back to Figure 1 at each stage (Section 2.1 - Section 2.5) to emphasize how design choices (datasets, models/modes, evaluators, and labeling) interact in a single, reproducible process.

2.1 Benchmarks and Tasks

We evaluate code generation across four public, execution-based benchmarks that validate functional correctness using unit tests: two "+" variants that expand test coverage (HumanEval+ and MBPP+) and two distributions of a large, realistic suite (BigCodeBench–Full and –Hard). HumanEval+/MBPP+ are provided by EvalPlus [17, 18], which also ships a standardized evaluation harness and sanitizer; BigCodeBench [19] provides both a complete set and a curated Hard subset with evaluation and post-processing tools. These choices allow us to probe a spectrum of task difficulty and test density while keeping evaluation consistent and reproducible.

2.2 Models and Inference Modes

We evaluate four API-served LLMs representative of both general-purpose and code-specialized systems—Qwen3-Plus [20], Qwen3-Coder-Plus [20, 21], DeepSeek-V3.1 [22], and Kimi-K2 [23]. All are modern, production-hosted families with long-context support, tool/agent capabilities, and standardized OpenAI-compatible endpoints, which enables uniform orchestration in our pipeline. Qwen3 offers a soft switch for reasoning ("thinking") vs. non-thinking behavior—exposed via enable_thinking and related API controls—allowing controlled within-model comparisons; DeepSeek-V3.1 implements a hybrid inference design ("Think/Non-Think—one model, two modes"); Kimi-K2 is a Mixture-of-Experts series emphasizing very long context and native tool calling. We use each model in its vendor-documented inference regime and keep decoding settings fixed for comparability.

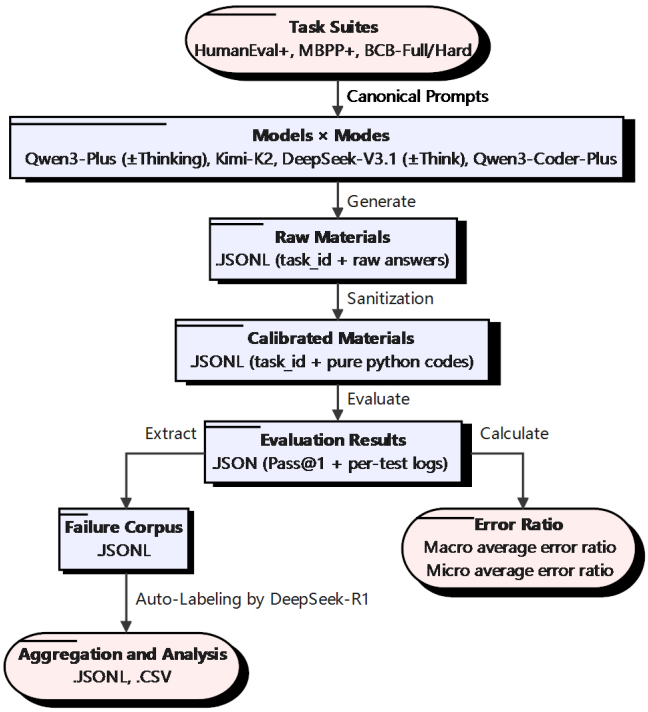


Figure 1: Experimental workflow

Concretely, Qwen3-Plus serves as a strong general model with configurable thinking controls through Alibaba Cloud Model Studio (OpenAI-compatible API); Qwen3-Coder-Plus is a code-oriented variant that adds engineering-centric features such as context caching for repeated inputs—useful for iterative code evaluation workflows. DeepSeek-V3.1 advertises 128K context, faster "thinking," and improved agent skills under its hybrid mode; we run both Think and Non-Think settings in generation. Kimi-K2 provides a trillion-parameter MoE with 256K context (platform docs) and tool-calling/agent integration, making it a natural counterpoint to the Alibaba/DeepSeek stack. Together, these choices span provider diversity, training focus (general vs. code-specialized), and inference style (with/without reasoning) while remaining accessible through stable, well-documented public endpoints.

The portfolio is designed to (i) compare reasoning vs. non-reasoning within the same model families where supported (Qwen3, DeepSeek-V3.1); (ii) contrast general vs. code-specialized training (Qwen3-Plus vs. Qwen3-Coder-Plus); and (iii) control for vendor and deployment differences (Alibaba Cloud, DeepSeek, Moonshot). This reduces the risk of overfitting conclusions to a single provider or decoding strategy and makes our error-composition and pass@1 analyses reproducible under the vendors' official APIs and documentation.

2.3 Generation & Evaluation Pipeline

We orchestrate all API calls on Alibaba Cloud, storing raw generations and metadata. (Provider details for Qwen are in the official Model Studio/API docs.)

For each benchmark task, we prompt the model with the canonical problem statement (function signature + natural-language description), request a single solution per task, and record the output. To quantify correctness, we compute Pass@1, the probability that at least one of the top-1 samples passes all tests—consistent with the original HumanEval definition and subsequent literature.

Before execution, we apply the benchmark-provided sanitizers to strip non-code artifacts and to normalize formatting/imports. We then execute the benchmark's unit tests in an isolated runner and log results (per-test verdicts, tracebacks, and failure messages). EvalPlus integrates these steps for HumanEval+/MBPP+, and BigCodeBench offers a dedicated sanitize utility and evaluation scripts—ensuring consistent, reproducible evaluation across datasets.

2.4 Error Taxonomy and Automated Labeling

From each benchmark run, we collect all failed tasks (solutions failing at least one unit test) and materialize them into a JSONL corpus containing: task identifier, prompt, model output (pre-/post-sanitization), failing test names, input/expected/got triples, and

error traces. This corpus is the input to our error analysis and labeling in error taxonomy and automated labeling.

We begin with the coarse outcome types exposed by the benchmark harnesses and refine them into fine-grained subtypes targeting logic defects. The three top-level categories (see section 3.2) we analyze most closely are: AssertionError / WrongAnswer (e.g., semantic mismatches discovered by assertions), ValueError (invalid values or domains at runtime) and TypeError (mismatched or unsupported types at runtime). Within each, we define concise sub-labels to capture the most frequent failure semantics observed in our corpus (Table 1). These sub-types can cover all error types under these three major categories without overlapping content. Harnesses from EvalPlus (HumanEval+/MBPP+) and BigCodeBench provide standardized sanitization and execution that surface these exception classes consistently across datasets, enabling uniform extraction of failure metadata for downstream labeling.

Given that the official error depends on the error that occurs in the first wrong test case (for example, if the first failed test case reports an AssertionError while the second failed test case in the same task reports a ValueError, then the task failure is judged as an AssertionError), when obtaining the JSONL files of failed tasks from the evaluation results, we retrieve and gather all the error types of failed test cases for each task and pick out the failed test cases of three top-level categories. We auto-annotate failures using DeepSeek-R1 in batch inference and the workflow is shown in Figure 2. For the reproducibility, we set the temperature as 0. DeepSeek-R1 is a dedicated reasoning model that emits chain-of-thought before final answers; we leverage this capacity to request a single best-fit fine-grained label (from our taxonomy) plus a brief justification, given the failing test name(s), I/O triplets, and traceback.

Our labeling is decoupled from the generation setting: the same taxonomy applies regardless of whether the underlying code was produced by V3.1 in Think or Non-Think mode, or by other models in our pool; this allows clean cross-mode comparisons of error composition. All failures are harvested after official sanitization and unit-test execution provided by EvalPlus and BigCodeBench, ensuring consistent mapping from raw exceptions to our top-level categories before fine-grained relabeling.

2.5 Error-Ratio Computation

We quantify failure severity on BigCodeBench by converting per-task unit-test results into error ratios and then aggregating them with macro and micro averages. All test counts are taken directly from the official test suites in the BigCodeBench dataset (Hugging Face), i.e., from the published unittest files associated with each task; no heuristics are used. BigCodeBench provides executable tests for each task and reports that tasks contain, on average, ∼5.6 test cases, enabling rigorous, test-driven evaluation.

Data flow. From a model's *_eval_results.json, we select tasks with at least one failing test (status = fail). For each such task, we load the corresponding official test file from the dataset and count all test methods (functions whose names start with test in the unittest class/module), obtaining the total number of tests N_t. We then read the evaluation log to identify the set of failed test names

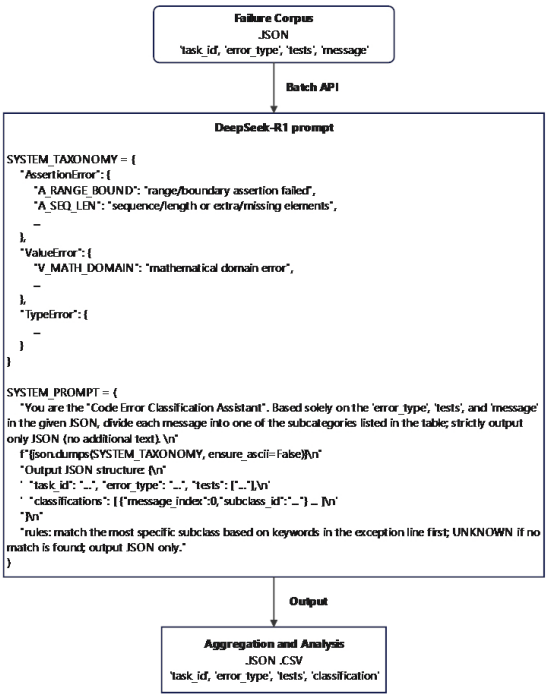


Figure 2: The workflow and prompt of batch inference using DeepSeek-R1

and compute the number of failures M_t. The per-task error ratio is

$$e_t = \frac{M_t}{N_t}.$$

Following the standard distinction in evaluation practice, we report both macro and micro averages, adapted here from classification-metric aggregation to the setting of per-task unit tests:

Macro average error ratio gives equal weight to each task:

$$E_{\text{macro}|} = \frac{1}{|F|} \sum_{t \in F} e_t,$$

where F is the set of failed tasks under consideration.

Micro average error ratio gives equal weight to each individual test case by pooling counts:

$$E_{\text{micro}|} = \frac{\sum_{t \in F} M_t}{\sum_{t \in F} N_t}$$

In other words, macro averages is $\frac{\#\text{problems with} \geq 1 \text{failing test}}{\#\text{problems evaluated}}$. Each problem counts once, regardless of how many asserts fail. Micro averages is $\frac{\sum \text{failing test asserts}}{\sum \text{executed test asserts}}$. Each test case is an instance; problems with more tests contribute more mass. The gap between them usefully indicates whether tasks with more test cases tend to be easier or harder (since micro is weighted by test count). All results

Table 1: Fine-grained error taxonomy used for automated labeling.

Category		Description	Example
AssertionError / WrongAnswer	A_EQ_NEQ	Equality/inequality assertion failed	assert [1–3] == [1–3] assert {"a":1} != {"a":1}
	A_SEQ_LEN	Sequence length or element set mismatch; extra/missing elements.	assert len([1, 2]) == 3 assert [0] == [0, 0]
	A_FLOAT_APPROX	Missing tolerance in floating-point comparison / numeric approximation.	assert 0.1 + 0.2 == 0.3
	A_NOT_NONE_EMPTY	None/emptiness predicate violated (expected non-empty or None).	items = [] assert items
	A_RANGE_BOUND	Range/boundary condition violated (off-by-one, interval endpoints).	x = 101 assert 0 <= x <= 100
	A_TYPE_CHECK	Type assertion failed (runtime type differs from expected).	assert isinstance("3", int)
	A_STRUCTURE_KEYS	Structural/key-set mismatch in dict/struct outputs.	expect = {"id": 1, "name": "A"} got = {"id": 1} assert expect == got
	A_MEMBERSHIP	Membership relation failed (e.g., x in S).	assert 3 in [1, 2]
	A_EXPECTED_EXCEPTION	Expected exception not raised or mismatched.	with self.assertRaises(ValueError): int("42")
ValueError	V_PARSE_CONVERT	Parsing/conversion failure (format errors, invalid casts).	int("abc") float("1.2.3")
	V_MATH_DOMAIN	Mathematical domain error (e.g., math.sqrt(-1) without complex).	import math math.sqrt(-1) math.log(0)
	V_UNPACK	Sequence unpacking arity mismatch.	a, b = [1–3]
	V_EMPTY_MINMAX	min/max on empty sequence.	max([])
	V_INVALID_OPTION	Invalid option/mode flag or unsupported configuration.	open("file.txt", mode="z")
TypeError	T_BAD_ARGTYPE	Argument type mismatch (e.g., int() on non-string/number).	int((1, 2)) len(5)
	T_NOT_ITERABLE	Object not iterable.	for x in 42: pass
	T_NOT_CALLABLE	Object not callable.	x = 5 x()
	T_ARGCOUNT	Positional/keyword argument count mismatch.	def f(a, b): pass f(1)
	T_UNSUPPORTED_OPERAND	Unsupported operand/compare types.	"a" + 1 3 < "3"
	T_UNHASHABLE	Unhashable object used as dict key / set element.	d = { [1, 2]: "x" }

are reproducible from the public BigCodeBench artifacts and their official test suites.

3 Results

3.1 Pass@1

We evaluate models using pass@1—the probability that a single generated program passes all unit tests—following the HumanEval convention and the practice adopted by recent code-LLM benchmarks. arXiv+1 For BigCodeBench we report pass@1 on both Full (1,140 tasks) and Hard (∼150 user-facing, more challenging tasks) as defined by the benchmark. bigcode-bench.github.io+1 The "+" variants (HumanEval+, MBPP+) are the EvalPlus extensions with much denser tests (≈80×/≈35×), which are known to lower nominal pass rates by exposing more faults.

Table 2: The whole results of all the models on all datasets with pass@1

Model	HumanEval	HumanEval+	MBPP	MBPP+	BCB(full)	BCB(hard)
DeepSeek-V3.1	0.927	0.884	0.899	0.741	0.499	**0.304**
DeepSeek-V3.1(Thinking)	0.951	0.902	0.963	0.810	**0.504**	0.284
Kimi-K2	0.939	0.915	0.910	0.751	0.474	0.270
Qwen-Plus	0.970	0.927	0.937	0.802	0.468	0.277
Qwen-Plus (Thinking)	**0.976**	**0.933**	**0.971**	**0.831**	0.495	0.284
Qwen3-Coder-Plus	0.939	0.884	0.923	0.788	0.496	0.297

The whole results are shown in the Table 2. Averaging across all six datasets, the ranking is: Qwen-Plus(Thinking) 0.748 > DeepSeek-V3.1(Thinking) 0.736 ≈ Qwen-Plus 0.730 > Qwen3-Coder-Plus 0.721 > Kimi-K2 0.710 ≈ DeepSeek-V3.1 0.709. Thus, adding a reasoning/"thinking" mode (where supported) improves the family average, but dataset composition strongly affects the leaderboard—particularly once test density and task realism increase (the "+" sets and BigCodeBench).

Averaging over all models, the mean pass@1 by dataset is: HumanEval: 0.950, MBPP: 0.934, HumanEval+: 0.908, MBPP+: 0.787, BigCodeBench (Full): 0.489 and BigCodeBench (Hard): 0.286. Two trends are clear: (i) EvalPlus "+" suites reduce pass@1 versus their base counterparts (mean drops: −4.3 pp for HumanEval→HumanEval+; −14.7 pp for MBPP→MBPP+), and (ii) BigCodeBench is substantially harder than HumanEval/MBPP—especially the Hard split. These observations are consistent with the design goals of EvalPlus (denser tests) and BigCodeBench (more realistic, user-facing tasks).

These patterns align with hybrid "Think/Non-Think" implementations exposed by the vendors, while highlighting that reasoning gains are not universal—they can diminish on the most challenging, user-facing tasks (BCB-Hard for V3.1). Therefore, thinking helps, but not always. Expect +2–3 pp average gains from thinking on these families, with possible regressions on the hardest, user-facing tasks (as seen on BCB-Hard for V3.1). Qwen's thinking mode shows uniform but smaller gains; DeepSeek's shows larger but less uniform gains.

3.2 Official Error Categories Across Datasets

The whole official error distribution is shown in Figure 3. Across all models and datasets, the official evaluators expose a consistent picture: failures concentrate in semantic buckets—WrongAnswer on EvalPlus suites and AssertionError on BigCodeBench—rather than in syntax or configuration faults. This aligns with the design intent of the benchmarks: HumanEval+/MBPP+ deliberately densify unit tests to surface functional mismatches, while BigCodeBench enforces specification compliance under rich tool/use and high branch coverage, naturally recording assertion-style breaks and a secondary mix of library/I/O exceptions.

Linking these patterns to our pass@1 findings, we observe that higher pass@1 on the base algorithmic suites coincides with fewer total failures but the same dominant error class—i.e., when models get more items correct, the remaining errors are still mostly WrongAnswer/AssertionError. Conversely, on BigCodeBench (Full/Hard), where pass@1 is lower, the evaluator logs a larger absolute volume of assertion-style failures plus a stable tail of environment/data–driven exceptions (e.g., key/attribute/missing-file), reflecting the benchmark's multi-library, tool-heavy construction rather than a shift toward syntax faults. In short, accuracy swings affect counts more than composition: the shape of official error categories is remarkably stable across settings.

Turning to reasoning ("thinking") modes, the comparison between non-thinking and thinking runs shows modest gains in pass@1 (notably on HumanEval/MBPP and their "+" counterparts), but little change in the evaluator-level mix: WrongAnswer/AssertionError remain the primary buckets. On BigCodeBench, thinking yields only small rebalancing of assertion shares and does not erase the characteristic tail of tool/I-O–related exceptions; the evaluator continues to attribute most residual failures to assertion mismatches, not syntax/runtime crashes. This is expected: thinking alters how models reason about a task but does not remove the coverage pressure applied by denser tests and multi-tool interactions.

3.3 Fine-grained Error Subclasses

The entire fine-grained error distribution is shown in Figure 4. Aggregating all failed cases under our taxonomy, we observe a single, robust pattern across benchmarks and models: comparison-semantics errors (A_EQ_NEQ) dominate the fine-grained landscape. In non-thinking runs, A_EQ_NEQ accounts for roughly the mid-40% to two-thirds of labeled failures, with the residual mass distributed among sequence/length mismatches, structural/key-set inconsistencies, expected-exception contracts, and value/parse options. This invariance holds despite sizeable differences in absolute failure counts implied by pass@1: when accuracy is higher (e.g., on base algorithmic suites), the number of failures shrinks but their composition remains anchored by comparison logic; when accuracy drops (e.g., on BigCodeBench Full/Hard), added failures largely preserve the same leading subclass while elevating dataset-specific tails. The pattern is consistent with benchmark design—EvalPlus stresses functional semantics via greatly denser unit tests, while BigCodeBench enforces specification compliance under rich library/tool use and very high coverage.

Enabling reasoning ("thinking") modes for the two families that support them (Qwen via enable_thinking; DeepSeek-V3.1 hybrid Think/Non-Think) rebalance but do not overturn these distributions. On the densified suites, A_EQ_NEQ contracts modestly, with compensating gains in sequence/length, numeric-tolerance, and parse/value subtypes; on BigCodeBench, the subclass mix is

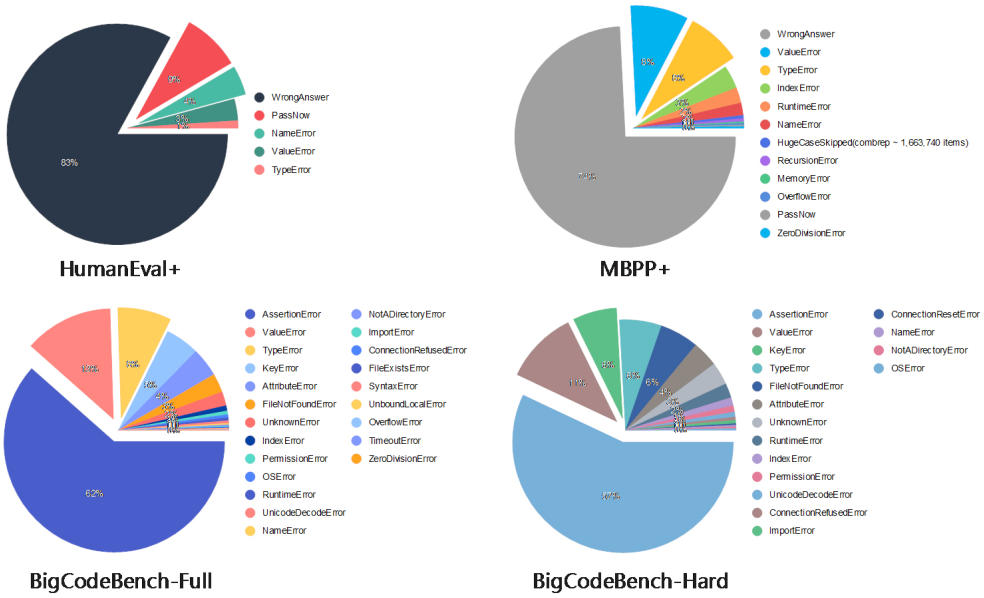


Figure 3: Official error distribution of all models (without thinking)

Table 3: Results of all models of macro error ratio and micro error ratio with pass@1

Model	BCB-fullPass@1	BCB-fullMacro Ratio	BCB-fullMicro Ratio	BCB-hardPass@1	BCB-hardMacro Ratio	BCB-hardMicro Ratio
DeepSeek-V3.1	0.499	0.558	0.544	**0.304**	0.521	0.514
DeepSeek-V3.1(Thinking)	**0.504**	0.560	0.548	0.284	**0.521**	**0.512**
Kimi-K2	0.474	0.572	0.558	0.270	0.538	0.523
Qwen-Plus	0.468	0.564	0.550	0.277	0.556	0.542
Qwen-Plus (Thinking)	0.495	**0.541**	**0.527**	0.284	0.538	0.525
Qwen3-Coder-Plus	0.496	0.555	0.542	0.297	0.544	0.532

essentially stable—A_EQ_NEQ remains the plurality and exception-contract/structure/parse clusters persist. In other words, thinking increases pass@1 but does not materially change what the evaluators expose as the fine-grained causes of residual failures: the principal bottleneck remains equality/inequality reasoning rather than syntax or configuration issues.

Taken together, these results argue that improvements visible in pass@1 predominantly scale counts, whereas the shape of failure is governed by benchmark objectives (dense semantic checks vs. tool-heavy, high-coverage tasks) and is remarkably stable across models and modes—aligning with recent empirical studies that identify logical/conditional mistakes as the primary source of code-LLM errors.

3.4 Pass@1 vs. Error Ratios

The results of error ratios on BigCodeBench datasets are in Table 3. Across both BigCodeBench full and hard subsets (the latter being a ~150-task, more user-facing and challenging slice of the full 1,140-task benchmark), Pass@1 tracks inversely with our error-ratio measures, as expected for unit-test–based functional evaluation. Models are ranked on BigCodeBench primarily by calibrated Pass@1, while our analysis complements this with macro and micro error ratios—respectively the unweighted mean of per-task error rates and the pooled failure rate over all test cases—following standard micro/macro aggregation conventions.

The table reveals a consistent difficulty gap between the full and hard sets: every model's Pass@1 declines on BCB-hard, in line with the subset's design, yet conditional failure severity (our error ratios computed on failed tasks) does not uniformly increase; in several cases it is slightly lower on BCB-hard, reflecting differences in test distributions and the conditioning on failure. Within this backdrop, DeepSeek-V3.1 exhibits an instructive trade-off when enabling deliberate reasoning ("Thinking"). On BCB-hard, Pass@1 decreases from 0.304 to 0.284, but both error ratios improve marginally (Macro: 0.521407→0.521107; Micro: 0.514334→0.511628), indicating fewer

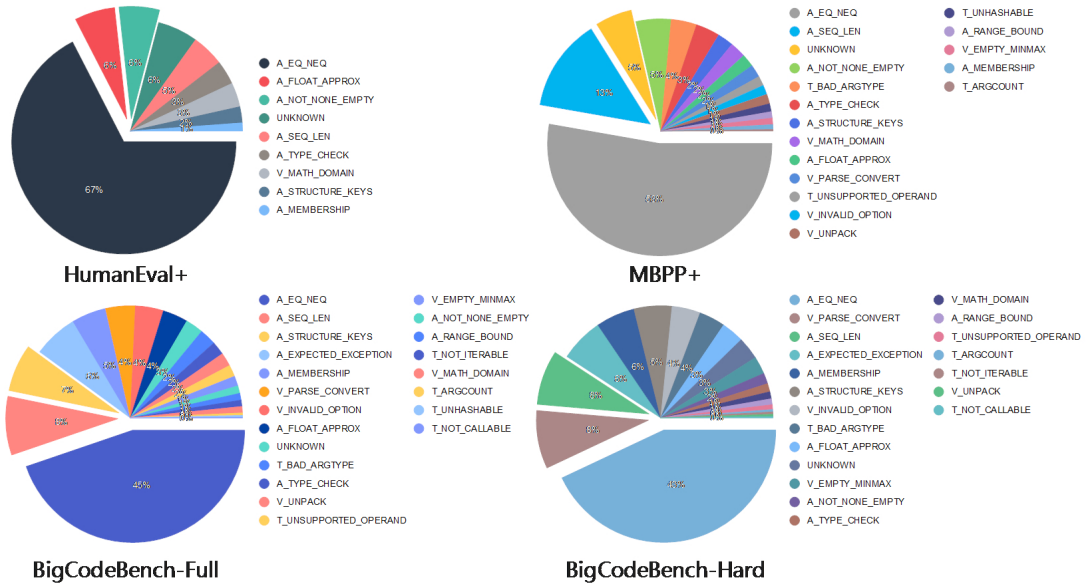


Figure 4: Fine-grained error distribution of all models (without thinking)

failed test cases per failed task even though the probability of obtaining a fully correct first attempt drops. This pattern is reversed on BCB-full, where Pass@1 increases slightly (0.499→0.504) but the error ratios rise (Macro: 0.558→0.560; Micro: 0.544→0.548), suggesting that deliberate prompting can shift the balance between first-try exactness and partial correctness in a dataset-dependent manner. Because Pass@1 is a "all-tests-must-pass" metric while error ratios measure failure severity over unit tests, such divergences are plausible and informative.

For the Qwen-Plus family, enabling deliberate reasoning meaningfully improves the profile. Without thinking, Qwen-Plus trails Qwen3-Coder-Plus in both Pass@1 and error ratios on BCB-full and BCB-hard. With thinking enabled, the Pass@1 gap narrows markedly (e.g., 0.468→0.495 on full, 0.277→0.284 on hard, approaching the coder model's 0.496/0.297), and the error ratios become lower than the coder variant (on full: Macro 0.541 vs 0.555 and Micro 0.527 vs 0.542; on hard: Macro 0.538 vs 0.544 and Micro 0.525 vs 0.532). This indicates that deliberate prompting in Qwen-Plus not only improves first-attempt success toward the coder-oriented model but also reduces the average number of failed unit tests among tasks that still fail, thus improving robustness at the test-case level.

We analyzed examples where the error rate of DeepSeek-v3.1 decreased after enabling deep thinking. In BigCodeBench/324, after enabling the deep thinking function, the test case test_process_still_running passed compared to when it was not enabled. The code after enabling deep thinking removed the exception handling, avoiding returning an error exit code of -1 during the running of the child process, thus allowing this test case to pass and reducing the number of failed test cases. This indicates that deep thinking can prompt the model to analyze problems more carefully and helps the model generate more robust code logic.

Meanwhile, we analyzed the phenomenon that after enabling deep thinking in DeepSeek-v3.1 for the same question, compared with the answers without deep thinking enabled, there were test cases that failed. In BigCodeBench/1004, after enabling deep thinking, the code directly uses zip(*top_10) to unpack the 10 most frequently occurring words. However, when the file is empty, top_10 is an empty list, causing zip(*top_10) to return an empty iterator. When unpacking, the code expects to get two variables (words and counts), but actually gets 0 values, thus triggering ValueError: not enough values to unpack (expected 2, got 0). Therefore, after enabling deep thinking, the model may tend to generate more concise and direct code, but it ignores the handling of boundary cases (such as empty input). This reflects that deep thinking may over-optimize the main logic in code generation at the expense of robustness. Although the code without deep thinking enabled may be more verbose, it covers more scenarios through explicit conditional checks and shows better error handling capabilities.

4 Conclusion

We introduced a unified error taxonomy and error-ratio metrics for evaluating code LLMs across multiple benchmarks. Our analysis reveals that semantic errors—particularly equality/inequality reasoning (A_EQ_NEQ)—form the primary bottleneck, even with deliberate reasoning. The error-ratio lens exposed critical insights invisible to Pass@k: DeepSeek-V3-1 (Thinking) achieved lower error ratios despite a drop in Pass@1, while Qwen-Plus (Thinking) narrowed its performance gap with a code-specialized model. This reproducible approach enables finer-grained progress measurement at the test-case level.

Limitations include our study's Python-only focus, fixed decoding parameters, and the potential biases of LLM-based batch inference labeling.

Future work should (i) deepen prompt-engineering for deliberate reasoning (e.g., self-consistency, structured intermediate plans, tool-aware scaffolds) to translate reductions in failure severity into higher Pass@k; (ii) improve training-data quality via targeted deduplication, contamination checks, and augmentation with verified unit-test corpora and tool-use traces; (iii) extend the failure-severity lens to other languages and task types while preserving rigorous test density and tool diversity; and (iv) report error-composition vectors and error-ratio curves alongside Pass@k to standardize multi-metric progress tracking as BigCodeBench and EvalPlus continue to evolve.

References

[1] Li R, Allal L B, Zi Y, *et al.* StarCoder: may the source be with you![A]. arXiv, 2023.
[2] Nijkamp E, Pang B, Hayashi H, *et al.* CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis[A]. arXiv, 2023.
[3] Fried D, Aghajanyan A, Lin J, *et al.* InCoder: A Generative Model for Code Infilling and Synthesis[A]. arXiv, 2023.
[4] OpenAI, Achiam J, Adler S, *et al.* GPT-4 Technical Report[A]. arXiv, 2024.
[5] Chen M, Tworek J, Jun H, *et al.* Evaluating Large Language Models Trained on Code[A]. arXiv, 2021.
[6] Chowdhery A, Narang S, Devlin J, *et al.* Palm: Scaling language modeling with pathways[J]. Journal of Machine Learning Research, 2023, 24(240): 1-113.
[7] Wen H, Zhu Y, Liu C, *et al.* Fixing code generation errors for large language models[J]. arXiv e-prints, 2024: arXiv-2409.
[8] Zhang S, Zhao H, Liu X, *et al.* NaturalCodeBench: Examining Coding Performance Mismatch on HumanEval and Natural User Prompts[A]. arXiv, 2024.
[9] Austin J, Odena A, Nye M, *et al.* Program Synthesis with Large Language Models[A]. arXiv, 2021.
[10] Zhuo T Y, Vu M C, Chim J, *et al.* BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions[A]. arXiv, 2025.
[11] Jain N, Han K, Gu A, *et al.* LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code[A]. arXiv, 2024.
[12] Lai Y, Li C, Wang Y, *et al.* DS-1000: A natural and reliable benchmark for data science code generation[C]//International Conference on Machine Learning. PMLR, 2023: 18319-18345.
[13] Wen H, Zhu Y, Liu C, *et al.* Fixing code generation errors for large language models[J]. arXiv e-prints, 2024: arXiv-2409.
[14] Wang Z, Zhou Z, Song D, *et al.* Where do large language models fail when generating code?[J]. arXiv e-prints, 2024: arXiv-2406.
[15] Zhong L, Wang Z, Shang J. Debug like a Human: A Large Language Model Debugger via Verifying Runtime Execution Step-by-step[A]. arXiv, 2024.
[16] Liu K, Koyuncu A, Kim D, *et al.* TBar: revisiting template-based automated program repair[C]//Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis. Beijing China: ACM, 2019: 31-42.
[17] Liu J, Xie S, Wang J, *et al.* Evaluating language models for efficient code generation[J]. arXiv preprint arXiv:2408.06450, 2024.
[18] Liu J, Xia C S, Wang Y, *et al.* Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation[J]. Advances in Neural Information Processing Systems, 2023, 36: 21558-21572.
[19] Zhuo T Y, Vu M C, Chim J, *et al.* BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions[A]. arXiv, 2025.
[20] Yang A, Li A, Yang B, *et al.* Qwen3 Technical Report[A]. arXiv, 2025.
[21] Hui B, Yang J, Cui Z, *et al.* Qwen2.5-Coder Technical Report[A]. arXiv, 2024.
[22] DeepSeek-AI, Liu A, Feng B, *et al.* DeepSeek-V3 Technical Report[A]. arXiv, 2025.
[23] Team K, Bai Y, Bao Y, *et al.* Kimi K2: Open Agentic Intelligence[A]. arXiv, 2025.